Informe Técnico

Simulación de Memoria Virtual con LMDB: Gestión de Grandes Volúmenes de Datos de Apuestas

Presentado por:

Jesús Alberto Mora Lozada
C.I.: 28.480.557

Universidad de Carabobo
Facultad Experimental de Ciencias y Tecnología
Arquitectura del Computador

Profesor: José Canache

Proyecto N.° 13: *Simulación de Memoria Virtual con LMDB*

Septiembre de 2026

Índice

Resumen **2**

1. El Problema **2**
1.1. Elección del dominio: apuestas de lotería 2
1.2. ¿Qué significa "cantidad masiva de datos"? 2

2. Objetivos **3**
2.1. Objetivo general 3
2.2. Objetivos específicos 3

3. Técnicas Computacionales para su Solución **3**
3.1. Marco teórico 3
3.1.1. Memoria virtual y jerarquía de memoria 3
3.1.2. Memoria secundaria y E/S 4
3.1.3. LMDB como caso de estudio de memoria virtual 4
3.1.4. Aclaratoria conceptual 4
3.2. Diseño del sistema 4
3.2.1. Modelo de datos 4
3.2.2. Esquema de claves y sub-bases 5
3.2.3. Dimensionamiento 6

4. Metodología de medición **6**

5. Análisis y Comparación de Resultados **7**
5.1. Generador (Fase 1) 7
5.2. Rendimiento de escritura (Fase 2) 8
5.3. Capa de consultas y validación (Fase 3) 8
5.4. Latencia y fallos de página (Fase 4) 9
5.5. Optimizaciones (Fase 5) 10
5.5.1. Hints de `madvise` 10
5.5.2. Escritura concurrente: el caso de las taquillas 10
5.5.3. Escalamiento del dataset 11

6. Plan de trabajo y criterios de éxito **11**
6.1. Fases 11
6.2. Criterios de éxito 12

7. Conclusiones **12**

8. Referencias **13**

Resumen

Este informe documenta el desarrollo de un sistema en C++ que usa **LMDB** (*Lightning Memory-Mapped Database*) para gestionar grandes volúmenes de datos como si fuera memoria secundaria, dentro del Proyecto N.° 13 de la asignatura Arquitectura del Computador. Como dominio de datos se eligió el de las **apuestas de lotería venezolana** (animalitos, terminales, tripletas y taquillas), por su riqueza de consultas y porque permite generar datasets sintéticos deterministas a escala arbitraria. El documento presenta el marco teórico de memoria virtual y jerarquía de memoria, el diseño del modelo de datos y del esquema de claves sobre el B+tree de LMDB, la metodología de medición, los resultados de las seis fases del proyecto (generador, benchmarks de escritura, capa de consultas, latencia y fallos de página, optimizaciones, y este informe final), y las conclusiones que vinculan las mediciones con la teoría de memoria virtual.

1. El Problema

El proyecto plantea: *"Simulación de Memoria Virtual con LMDB: usar LMDB para gestionar grandes volúmenes de datos como si fuera memoria secundaria. Optimizar consultas y escritura. Medir tiempo de acceso y latencia. Manejo de grandes volúmenes de datos."* El enunciado sugiere como ejemplo datasets de áreas sísmicas, pero el objetivo es agnóstico al dominio: lo que importa es que el volumen de datos sea lo bastante grande como para que la paginación del sistema operativo sea observable y medible.

1.1. Elección del dominio: apuestas de lotería

Se eligió el dominio de las apuestas de lotería venezolana por tres razones:

1. **Perfil de acceso ideal para LMDB.** Los datos sísmicos son pocos archivos enormes con acceso mayormente secuencial; las apuestas son miles de millones de registros pequeños con consultas puntuales y por rango, que ejercitan exactamente lo que LMDB hace bien: búsquedas por clave en B+tree, cursores y barridos por rango.

2. **Consultas con sentido de negocio.** Frecuencia de un animalito por lotería, rachas (cuánto lleva un animalito sin salir), terminales calientes, agregados por hora y por taquilla.

3. **Relevancia práctica.** El autor desarrolla en paralelo un sistema de apuestas real; este proyecto sirve como ensayo de dimensionamiento y de decisiones de diseño aplicables a ese sistema si llegara a escalar (por ejemplo, a 10 000 taquillas).

1.2. ¿Qué significa "cantidad masiva de datos"?

No es un número absoluto de gigabytes: para este proyecto, *masivo* significa que el dataset **no cabe en la memoria principal** y el sistema operativo se ve obligado a

paginar, con fallos de página medibles. En una máquina típica de 8–16 GB de RAM eso se consigue con datasets de decenas de gigabytes. La Sección 3.2.3 calibra esa cifra con la medición real de la Fase 1: ≈ 180 B por jugada en disco.

2. Objetivos

2.1. Objetivo general

Implementar en C++ un sistema sobre LMDB que gestione grandes volúmenes de datos de apuestas como si fuera memoria secundaria, optimizando escrituras y consultas y midiendo tiempo de acceso y latencia, para demostrar en la práctica el mecanismo de memoria virtual (paginación por demanda) del computador.

2.2. Objetivos específicos

1. Diseñar un modelo de datos y un esquema de claves binarias que exploten el orden del B+tree de LMDB (claves big-endian, índices secundarios, MDB_APPEND).
2. Construir un generador sintético determinista que produzca datasets de apuestas de tamaño configurable (de cientos de MB a decenas de GB).
3. Medir el throughput de escritura masiva y la latencia de consultas puntuales y por rango (p50/p95/p99), incluyendo fallos de página mayor y menor.
4. Demostrar la diferencia de latencia entre acceso con caché tibia (dato en RAM) y caché fría (dato traído de disco por demanda de páginas).
5. Validar el diseño bajo un escenario de carga realista: 10 000 taquillas generando jugadas de forma sostenida.
6. Relacionar todas las mediciones con la teoría de memoria virtual del curso (Capítulos 5 y 6 de Patterson y Hennessy).

3. Técnicas Computacionales para su Solución

3.1. Marco teórico

3.1.1. Memoria virtual y jerarquía de memoria

El sistema de memoria virtual traduce direcciones virtuales a direcciones físicas y divide ambos espacios en páginas; la traducción se acelera con la TLB. Cuando un proceso accede a una página que no está en memoria principal, ocurre un **fallo de página**: el sistema operativo la trae de la memoria secundaria (disco o SSD) y reanuda la instrucción. Es la paginación por demanda: el disco se comporta como una extensión de la RAM, con el costo de latencia que eso implica (Patterson y Hennessy, §5.4, pág. 492).

Dos conceptos del mismo capítulo gobiernan el rendimiento de nuestro sistema:

- **Localidad espacial y temporal** (§5.2–5.3): los accesos a datos cercanos y recientes se resuelven en niveles altos de la jerarquía; un buen diseño de claves agrupa en páginas contiguas los datos que se consultan juntos.
- **Fallos de página mayor y menor**: el fallo menor resuelve con datos ya en el caché de páginas del sistema; el mayor implica I/O real al dispositivo. Medir ambos distingue RAM de disco.

3.1.2. Memoria secundaria y E/S

Los datos de LMDB viven en disco o SSD, así que la latencia de un fallo mayor depende de la tecnología del dispositivo: discos magnéticos (§6.3, pág. 575) con latencias de milisegundos por búsqueda, frente a memorias Flash (§6.4, pág. 580) con latencias mucho menores y lectura aleatoria mucho más barata. La metodología de medición de prestaciones de E/S (§6.7, pág. 596) es la base de nuestros benchmarks.

3.1.3. LMDB como caso de estudio de memoria virtual

LMDB no implementa un caché propio como otros motores: **mapea el archivo de la base de datos al espacio de direcciones virtuales del proceso** (`mmap`). El caché de páginas del sistema operativo *es* el caché: una lectura a un dato ya residente en RAM es un acceso directo a memoria (sub-microsegundo); un dato no residente dispara un fallo de página y se paga la latencia del dispositivo. Otras propiedades relevantes del motor:

- **Estructura B+tree copia-en-escritura**: lecturas sin bloqueos (MVCC); un solo escritor por transacción.
- **Claves comparadas byte a byte** (`memcmp`): el ordenamiento de claves es lexicográfico, lo que obliga a codificar los enteros multi-byte en big-endian para que el orden en disco coincida con el orden numérico.
- MDB_APPEND: inserción masiva en el borde derecho del árbol, mucho más barata que las inserciones aleatorias, si las claves llegan ordenadas.

3.1.4. Aclaratoria conceptual

La sección §5.6 del libro (pág. 525) trata de *máquinas virtuales* (hipervisores y virtualización de hardware), un concepto distinto de la *memoria virtual*. En este informe el término se usa siempre en el segundo sentido.

3.2. Diseño del sistema

3.2.1. Modelo de datos

El dataset modela un negocio de apuestas: varias loterías (*Lotto Activo*, *La Granjita*, *Guácharo Activo*, *Selva Plus*, *Lotería de Caracas*, *Lotto Rey*) celebran sorteos a horas fijas del día; las **taquillas** (puntos de venta) registran **jugadas** de tres tipos:

- **Animalito**: jugada a una figura (37 figuras, ids 0–36).
- **Terminal**: jugada a una terminación de dos dígitos (00–99).
- **Tripleta**: jugada a tres figuras sin importar el orden.

Cada jugada guarda: lotería, marca de tiempo, taquilla, tipo, selección y monto jugado. Cada sorteo guarda el animalito que salió.

3.2.2. Esquema de claves y sub-bases

Un único entorno LMDB contiene cinco sub-bases con nombre (Tabla 1). Las claves son binarias de largo fijo; todo entero multi-byte en claves va **big-endian** para que el orden lexicográfico coincida con el numérico. La clave maestra empieza por lotería y sigue con la marca de tiempo, de modo que el flujo de claves generado es estrictamente ascendente y permite MDB_APPEND.

Cuadro 1: Sub-bases del entorno LMDB.

Base	Propósito	Clave	Valor
`bets`	registro maestro (ascendente)	13 B: lotería, ts, taquilla, seq	16 B: tipo, sel
`i_animalito`	frecuencia y rachas	16 B: lotería, animalito, ts, taquilla, seq	8 B: monto
`i_terminal`	frecuencia de terminales	16 B: lotería, terminal, ts, taquilla, seq	8 B: monto
`i_taquilla`	replay por punto de venta	16 B: taquilla, ts, lotería, seq	8 B: monto
`draws`	resultados de sorteos	6 B: lotería, día, sorteo	2 B: animalito

El Listado 1 muestra la codificación de la clave maestra; el comentario documenta la regla big-endian que garantiza los barridos por rango.

Listing 1: Codificación de la clave maestra (src/keys.h).

```
// La clave arranca por loteria y sigue con la marca big-endian: el orden
// lexicografico de los bytes coincide con el orden numerico.
inline std::array<uint8_t, kBetsKeySize> encode_bets_key(uint8_t lottery,
                                                         uint64_t ts,
                                                         uint16_t taquilla,
                                                         uint16_t seq) {
  std::array<uint8_t, kBetsKeySize> k{};
  size_t o = 0;
  k[o++] = lottery;
```

```
  put_be64(&k[o], ts);    // 8 bytes big-endian
  o += 8;
  put_be16(&k[o], taquilla);
  o += 2;
  put_be16(&k[o], seq);
  return k;
}
```

3.2.3. Dimensionamiento

La Fase 1 midió ≈ 180 B por jugada en disco para el dataset de humo, incluyendo las tres entradas de índice y la sobrecarga del B+tree. Esa constante baja con la escala (la sobrecarga del B+tree se amortiza): a 100 M de jugadas el costo medido es ≈ 163 B por jugada.

Cuadro 2: Escala del dataset en función del número de jugadas.

Jugadas	**Tamaño**
1 M (humo)	180 MB (medido)
10 M	1,6 GB (medido)
50 M	≈ 8 GB
100 M	**15,5 GB (medido)**
500 M	≈ 78 GB

Para cerrar el criterio de escala, se generó y validó el dataset de **100 M de jugadas**: 100 000 000 jugadas en 16 280 166 400 B (≈ 15,5 GB), con agregados exactos (**149/149** comprobaciones) y una altura del B+tree de 4 niveles para `bets`. Es el punto de operación principal del experimento.

En una máquina de 8–16 GB de RAM, 15,5 GB no caben y la paginación por demanda es inevitable (ese es el escenario que el proyecto busca demostrar). En la máquina de pruebas de este informe (32 GB de RAM) el dataset *sí* cabe; por eso el efecto frío no se obtiene por presión de RAM sino forzándolo con `madvise`, lo que permite medir la paginación por demanda de forma controlada y reproducible.

4. Metodología de medición

Todas las mediciones se tomaron sobre el mismo equipo: un procesador Intel Core i7-9750H (6 núcleos, 2,60 GHz), 32 GB de RAM y un SSD NVMe Samsung PM981, con los datasets almacenados en el SSD. La latencia de un fallo de página mayor corresponde por tanto a un SSD NVMe, no a un disco magnético (véanse §6.3 y §6.4 del libro). Las cifras frías dependen del medio de almacenamiento; en un disco magnético serían aún más costosas.

Las métricas de las fases 3 y 4 son:

- **Latencia por operación**: percentiles p50, p95 y p99 de lookups puntuales y de barridos por rango, con reloj monotónico.
- **Throughput**: operaciones por segundo para lecturas secuenciales y escrituras por lotes.
- **Fallos de página**: contadores mayor/menor medidos con la diferencia de `/proc/self/stat` antes y después de cada corrida, correlacionados con el tamaño del dataset y el tamaño de la RAM.
- **Caché fría vs. tibia**: la misma consulta inmediatamente después de poblar el caché (tibia) frente a tras invalidar el caché de páginas con `madvise(MADV_DONTNEED)` sobre la región mmap del archivo LMDB (fría, sin privilegios de root). La brecha es exactamente el costo del fallo de página mayor.
- **Escenario 10k taquillas**: escritura sostenida equivalente a 10 000 taquillas ($\approx$ 1,7 k jugadas/s) con latencia de escritura p99 acotada.

5. Análisis y Comparación de Resultados

5.1. Generador (Fase 1)

El generador produce, en una sola corrida determinista, los sorteos y las jugadas de todas las loterías y escribe el manifiesto con la verdad absoluta del dataset (conteos por lotería, tipo, animalito y terminal; estadísticas del B+tree por sub-base; tamaño y duración). Dos garantías verificadas:

- **Determinismo**: al regenerar el dataset de humo con la misma semilla tras refactorizar el código, el archivo resultó byte a byte idéntico (188,260,352 B).
- **Orden de claves**: una auto-prueba camina el B+tree con un cursor y verifica que las marcas de tiempo no descienden dentro de cada lotería (codificación big-endian correcta).

Cuadro 3: Resultados de la prueba de humo (1 M de jugadas).

Métrica	**Valor**
Jugadas generadas	1 000 000 (validación de agregados: aprobada)
Sorteos	37 230 (6 loterías $\times$ 3 años)
Tamaño del archivo	188 260 352 B ($\approx$ 180 MB)
Duración	$\approx$ 3 s (reloj de pared, esta máquina)
Mezcla de tipos	animalito 75,0 % · terminal 20,0 % · tripleta 5,0 %
Auto-prueba de orden	aprobada

La distribución de popularidad de los animalitos sigue una ley tipo Zipf (el animalito 0 concentra $\approx$ 12 % de las jugadas de animalito), lo que da estructura real a las consultas de frecuencia y de rachas de las fases siguientes.

5.2. Rendimiento de escritura (Fase 2)

El benchmark de escritura mide el mismo patrón de lotes que usa el generador (una transacción de escritura cada 50 k registros) en dos modos: claves ascendentes con MDB_APPEND y claves aleatorias con puts normales (Tabla 4).

Cuadro 4: Benchmark de escritura: 1 M de registros de 104 B (clave 8 B + valor 96 B), lotes de 50 k registros.

Métrica	**Append (ordenado)**	**Aleatorio**	**Razón**
Registros/s	1,94 M	226 k	×8,6
MB/s	192	22,4	×8,6
Commit p50	11,6 ms	101 ms	×8,7
Tamaño final	118 MB	398 MB	×3,4

La lección que queda para el informe final: el orden de las claves no es cosmético. Con claves ascendentes cada inserción cae en el borde derecho del B+tree y una misma hoja se llena de corrido; con claves aleatorias cada inserción abre o divide páginas dispersas: el archivo final queda 3,4× más grande y el rendimiento cae 8,6×.

5.3. Capa de consultas y validación (Fase 3)

Se implementó la herramienta de consultas sobre el dataset, con ocho comandos:

- `info`: estadísticas del entorno y de cada sub-base del B+tree (entradas, hojas, ramas, profundidad).
- `lookup`: búsqueda puntual por clave exacta. Ejemplo real medido: la clave $(0, 1577838271, 46, 0)$ devuelve una jugada de animalito con monto 7 752 céntimos.
- `range`: barrido por rango de fechas dentro de una lotería (primera semana de la lotería 0: 1 065 jugadas).
- `animalito-stats`: frecuencia y monto por animalito usando el índice; la popularidad Zipf queda a la vista (animalito 0 con 20 592 jugadas en la lotería 0 frente a 3 669 del animalito 11).
- `terminal-stats`: frecuencia y monto por terminal.
- `racha`: días que lleva un animalito sin salir en los sorteos (el animalito 5 de la lotería 0 salió 272 veces y lleva 10 días sin salir).
- `taquilla`: replay completo de un punto de venta (la taquilla 7 registró 19 873 jugadas).
- `validate`: validación cruzada de TODOS los agregados contra el manifiesto: **149/149 comprobaciones APROBADAS**.

5.4. Latencia y fallos de página (Fase 4)

Se implementaron tres modos de lectura en la herramienta de benchmarks (`bin/bench`), cada uno en dos variantes: *tibio* (páginas en RAM) y *frío* (páginas liberadas antes de medir). La liberación se hace con `madvise(MADV_DONTNEED)` sobre la región `mmap` del archivo LMDB, encontrada en `/proc/self/maps`—sin necesidad de privilegios de root. Los fallos de página se miden con la diferencia de contadores en `/proc/self/stat` (campo `minflt` y `majflt`).

- `read-lookup`: N búsquedas puntuales sobre claves existentes muestreadas del propio dataset (hasta 100 k claves muestreadas con zancada uniforme).
- `read-scan`: barrido secuencial completo de todas las jugadas de una lotería.
- `read-index`: barrido secuencial completo del índice `i_animalito`.

Los resultados se midieron sobre el dataset de humo (1 M de jugadas, ≈ 180 MB) en una máquina con 32 GB de RAM (Tabla 5).

Cuadro 5: Benchmarks de lectura: tibio vs. frío (dataset de 1 M de jugadas, ≈ 180 MB).

Modo	**Tibio ops/s**	**Frío ops/s**	**Factor**	**Frío majflt**	**Frío p99**
read-lookup (200 k)	1 156 332	314 544	×3,7	68	4,3 μs
read-scan (167 k)	8 428 337	1 150 678	×7,3	11	0,078 μs
read-index (900 k)	1 866 943	1 499 141	×1,2	76	0,072 μs

Análisis de los resultados:

- **Lookups puntuales**: la latencia tibia es sub-microsegundo (p50 = 0,81 μs), consistente con una búsqueda en B+tree cuyas páginas están en RAM. El modo frío multiplica la latencia p99 por ×3,7 (de 1,3 μs a 4,3 μs) y genera 68 fallos de página mayores—cada fallo trae una página de 4 KB desde el dispositivo. El conjunto de trabajo de 200 k lookups es compacto (pocas páginas del B+tree), por lo que el efecto frío es moderado.
- **Barrido secuencial**: el barrido tibia alcanza 8,4 M ops/s porque el kernel detecta la secuencia y activa la relectura anticipada (*readahead*); en modo frío solo 11 fallos mayores porque la relectura anticipada trae las páginas antes de que se necesiten. La reducción de throughput (×7,3) se debe al costo inicial de las primeras páginas.
- **Índice**: el índice `i_animalito` (900 k entradas, ≈ 23 MB) cabe casi en su totalidad en RAM incluso tras la liberación; la diferencia tibio/frío es mínima (×1,2).

Estos resultados confirman la predicción teórica: el costo de un fallo de página mayor es dominante en accesos aleatorios (lookups), mientras que los accesos secuenciales amortizan ese costo con la relectura anticipada.

5.5. Optimizaciones (Fase 5)

Se exploraron dos estrategias de optimización:

5.5.1. Hints de `madvise`

Se aplicaron hints al kernel para anunciar el patrón de acceso esperado: `MADV_RANDOM` antes de lookups puntuales y `MADV_SEQUENTIAL` antes de barridos. Los resultados sobre el dataset de 1 M (Tabla 6) muestran un efecto contraintuitivo:

Cuadro 6: Efecto de los hints de `madvise` sobre el dataset de 1 M (modo frío).

Modo	Sin hint ops/s	Con hint ops/s	Con hint majflt
read-lookup frío	314 544	50 217	9 304
read-scan frío	1 150 678	1 284 638	12

- `MADV_RANDOM` empeora los lookups fríos (×6,3 más lento): al desactivar la relectura anticipada, cada lookup frío produce su propio fallo mayor sin amortización. Sin el hint, el kernel "adivina" correctamente y trae páginas adyacentes que reducen los fallos de 9 304 a 68.
- `MADV_Sequential` mejora marginalmente el barrido frío (+12 %): el kernel ya detecta el acceso secuencial por defecto; el hint solo amplía la ventana de relectura.

La lección: **no desactivar la relectura anticipada del kernel es en sí mismo una optimización**, porque el kernel suele "adivinar" mejor que una política estática.

5.5.2. Escritura concurrente: el caso de las taquillas

Se implementó un benchmark que simula N taquillas escribiendo en paralelo sobre un mismo entorno LMDB. LMDB solo permite **un escritor a la vez** (su constraint fundamental); todos los hilos comparten un mutex que serializa las transacciones de escritura. El benchmark mide cuánto se degrada el rendimiento con N taquillas compitiendo por el escritor (Tabla 7).

Cuadro 7: Escritura concurrente: 500 000 registros por taquilla, lotes de 50 000.

Taquillas	Total	Throughput	Commit p50	Degradación
1	500 000	1 342 000 reg/s	11,2 ms	—
4	2 000 000	1 104 000 reg/s	21,8 ms	×0,82
8	4 000 000	1 004 000 reg/s	24,3 ms	×0,75
16	8 000 000	943 000 reg/s	26,3 ms	×0,70

El throughput cae ×0,70 al pasar de 1 a 16 taquillas, y el commit p50 se duplica (de 11,2 ms a 26,3 ms). Esto confirma que el cuello de botella de LMDB bajo carga

concurrente es el **escritor único**: cada taquilla debe esperar a que la anterior termine su transacción. En un sistema real con 10 000 taquillas, la solución es partitionar los datos (por rango de tiempo o por taquilla) en múltiples entornos LMDB, o usar un proceso recolector que agrupe las escrituras de múltiples taquillas en lotes grandes antes de persistir.

5.5.3. Escalamiento del dataset

Se generó un dataset de 10 M de jugadas (≈ 1,57 GB) para observar el efecto frío con mayor claridad (Tabla 8).

Cuadro 8: Benchmarks de lectura sobre 10 M de jugadas (≈ 1,57 GB).

Modo	**Tibio ops/s**	**Frío ops/s**	**Factor**	**Frío majflt**
read-lookup (200 k)	907 327	3 400	×267	83 251
read-scan (1,67 M)	2 029 871	1 414 438	×1,4	77

El efecto frío en lookups ahora es **devastador**: factor ×267 de lentitud, con 83 251 fallos mayores (cada uno trae 4 KB desde el dispositivo). Esto confirma que con un dataset que excede la RAM, la paginación por demanda domina el rendimiento de accesos aleatorios. Los barridos secuenciales siguen siendo resilientes (×1,4) gracias a la relectura anticipada del kernel.

Las cifras de *fallos de página* son muy estables entre corridas (±0,3 %), mientras que el *throughput* frío depende del estado del disco y del caché en el momento de la medición; por eso conviene leerlas como órdenes de magnitud, no como valores exactos.

6. Plan de trabajo y criterios de éxito

6.1. Fases

1. Setup y generador determinista — **completada**.
2. Benchmarks de carga masiva y throughput de escritura — **completada**.
3. Capa de consultas: lookups puntuales, barridos por rango, consultas por índice (frecuencia, rachas, taquillas) — **completada**.
4. Benchmarking de latencia y fallos de página (frío vs. tibio) — **completada**.
5. Iteraciones de optimización guiadas por las mediciones — **completada**.
6. Informe final con la conexión explícita a la teoría de memoria virtual — **completada**.

6.2. Criterios de éxito

Cuadro 9: Criterios de éxito del proyecto.

Área	**Criterio**
Funcional	Dataset de 100 M de jugadas cargado con agregados exactos (149/149); $\geq$ 4 tipos de consulta correctos contra el manifiesto.
Rendimiento	Lookup puntual tibio $\leq 10\ \mu$s en p99; lecturas secuenciales $\geq$ 500 k ops/s; escritura $\geq$ 100 k jugadas/s.
Memoria virtual	Brecha frío/tibio medida y documentada; fallos de página mayor/menor reportados por consulta.
Escenario real	Comportamiento de escritura concurrente medido de 1 a 16 taquillas (caída $\times$0,70); estrategia de escalado hacia 10 000 taquillas derivada de la medición.
Académico	Informe que vincula *mmap* con la paginación por demanda usando mediciones propias, no teoría copiada.

7. Conclusiones

- El dominio de apuestas demuestra ser adecuado: registros pequeños y numerosos, claves ordenables por tiempo y consultas con estructura de negocio real.
- La decisión de diseño central de la Fase 1 — claves big-endian con flujo ascendente — habilita MDB_APPEND y los barridos por rango, y quedó verificada con una auto-prueba y con determinismo byte a byte.
- La constante medida de $\approx$ 180 B por jugada (que baja a $\approx$ 163 B a escala) convierte el objetivo de "datos masivos" en un número accionable: 100 M de jugadas = 15,5 GB, generadas y validadas (149/149), que garantiza paginación observable en hardware de estudiante.
- El benchmark de escritura demostró que ordenar las claves da $\times$8,6 de rendimiento y $\times$3,4 de espacio — una lección de diseño de B+tree que se aplica a cualquier base de datos.
- La validación de 149/149 comprobaciones contra el manifiesto demuestra que la capa de consultas es correcta y reproducible.
- Los benchmarks de lectura confirmaron la predicción teórica: los accesos aleatorios (lookups) son $\times$267 más lentos en caché fría que en tibia para un dataset de 10 M de jugadas ($\approx$ 1,6 GB); los accesos secuenciales apenas se afectan ($\times$1,4) gracias a la relectura anticipada del kernel.
- La optimización con hints de `madvise` demostró que **no interferir** con la relectura anticipada del kernel es en sí mismo una optimización: `MADV_RANDOM` empeora los

lookups fríos porque desactiva una "adivinanza" que el kernel hace mejor que una política estática.

- La técnica de caché fría con `madvise(MADV_DONTNEED)` sobre la región mmap del archivo LMDB funciona sin privilegios de root, lo que la hace apta para entornos académicos donde el estudiante no tiene acceso a `sudo`.

8. Referencias

1. Patterson, D. A. y Hennessy, J. L. (2014). *Estructura y diseño de computadores: la interfaz hardware/software* (4.ª ed.). Reverté. Capítulos 5 ("Grande y rápida: aprovechamiento de la jerarquía de memoria", en particular §5.4 "Memoria virtual", pág. 492) y 6 ("Almacenamiento y otros aspectos de la E/S").

2. OpenLDAP Foundation. *LMDB: Lightning Memory-Mapped Database* — documentación técnica. https://www.lmdb.tech/doc/

3. `mmap(2)` — páginas del manual de Linux.